



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Домашнее задание № 2
по курсу «Моделирование»
«Построение вероятностной модели»

Студент группы ИУ9-82Б Морозов Д.В.

Преподаватель Домрачева А.Б.

Москва 2026

1 Цель и постановка задачи

Целью данной домашней работы является построение вероятностной модели для поставленной задачи.

Постановка задачи: За одной из N дверей находится приз, за остальными - пусто. Игроку предлагается выбрать одну из N дверей наудачу. Не открывая выбранной двери и не показывая двери с призом, ведущий открывает K пустых дверей по своему желанию. В завершение игры игрок может поменять или сохранить свой выбор. Необходимо провести вычислительный эксперимент, изменяя значения K , N , а также вероятность p сохранения/изменения выбора на завершающем этапе игры с целью оценивания вероятности выигрыша игрока.

Необходимо реализовать модель программно и проверить ее адекватность решением задачи, известной как парадокс Монти Холла, и сравнить результаты вычисления по построенной модели с результатами вычислительного эксперимента для $K=1,2,\dots, N-2$. Количество дверей N может меняться от 3 до 10.

2 Построение математической модели

Для построения вероятностной модели необходимо определить параметры задачи и результат, который требуется вычислить. Входными данными модели являются следующие параметры:

- N — общее количество дверей в игре. За одной дверью находится приз, за остальными $N - 1$ дверями находятся пустые ячейки. В рассматриваемой задаче количество дверей может изменяться в диапазоне от 3 до 10.
- K — количество пустых дверей, которые открывает ведущий после выбора игрока. При этом выполняется условие $1 \leq K \leq N - 2$, поскольку ведущий не открывает дверь игрока и дверь с призом.
- p — вероятность того, что игрок сохраняет первоначальный выбор двери на завершающем этапе игры.
- $1 - p$ — вероятность того, что игрок изменяет свой выбор и выбирает одну из оставшихся закрытых дверей.

Выходной величиной модели является вероятность выигрыша игрока P_{win} , то есть вероятность того, что в результате игры игрок выберет дверь, за которой находится приз.

В данной задаче игрок сначала случайным образом выбирает одну из N дверей. На этом этапе вероятность того, что он сразу выбрал дверь с призом, равна $\frac{1}{N}$. После выбора игрока ведущий открывает K пустых дверей. При этом ведущий знает, где находится приз, и поэтому не открывает дверь с призом и дверь, выбранную игроком. Таким образом, действия ведущего не являются случайными — они зависят от того, где на самом деле находится приз. Следовательно, информация о том, какие двери открыл ведущий, связана с расположением приза. Это означает, что после открытия дверей вероятность того, что игрок изначально выбрал правильную дверь, может измениться.

Чтобы формально учесть эту зависимость и пересчитать вероятность события после получения новой информации, воспользуемся формулой Байеса для формализации нашей вероятностной модели. Она позволяет определить вероятность события A при условии, что произошло событие B .

В нашей задаче событие A означает, что игрок изначально выбрал дверь с призом, а событие B — что ведущий открыл K пустых дверей. Тогда вероятность правильности первоначального выбора после открытия дверей определяется как

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}.$$

На рисунке 1 приведена схема вероятностной модели.

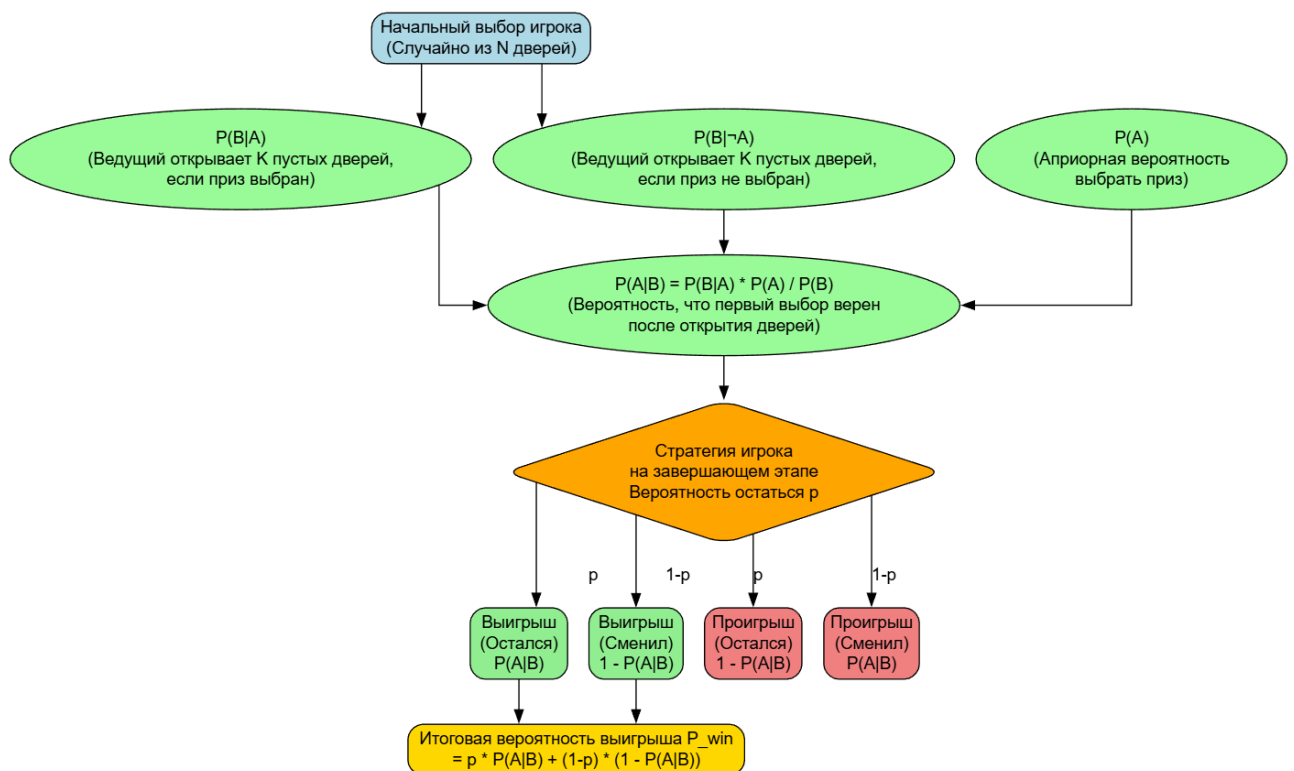


Рис. 1 — Схема вероятностной модели

3 Решение вычислительной задачи

Рассмотрим вероятности, входящие в формулу Байеса. Для этого проанализируем действия ведущего в двух возможных ситуациях: когда игрок изначально выбрал правильную дверь и когда он выбрал пустую дверь.

Если игрок сразу выбрал дверь с призом (событие A), то все остальные $N - 1$ дверей являются пустыми. В этом случае ведущий может открыть любые K дверей из этих $N - 1$ пустых дверей. Таким образом, вероятность события B при условии события A равна

$$P(B|A) = \frac{1}{C_{N-1}^K},$$

где C_{N-1}^K — число способов выбрать K дверей из $N - 1$ пустых.

Если же игрок изначально выбрал пустую дверь (событие $\bar{A}$), то среди оставшихся $N - 1$ дверей одна содержит приз, а остальные $N - 2$ являются пустыми. Поскольку ведущий не может открыть дверь с призом, он выбирает K дверей только из $N - 2$ пустых дверей. Вероятность события B в этом случае равна

$$P(B|\bar{A}) = \frac{1}{C_{N-2}^K},$$

где C_{N-2}^K — число способов выбрать K дверей из $N - 2$ пустых.

Используя формулу полной вероятности, получаем

$$P(B) = P(B|A)P(A) + P(B|\bar{A})P(\bar{A}) = \frac{1}{C_{N-1}^K} \cdot \frac{1}{N} + \frac{1}{C_{N-2}^K} \cdot \frac{N-1}{N}.$$

Вероятность того, что игрок изначально выбрал правильную дверь после открытия K дверей ведущим, определяется по формуле Байеса:

$$P(A|B) = \frac{P(B|A) P(A)}{P(B|A) P(A) + P(B|\bar{A}) P(\bar{A})} = \frac{\frac{1}{C_{N-1}^K} \cdot \frac{1}{N}}{\frac{1}{C_{N-1}^K} \cdot \frac{1}{N} + \frac{1}{C_{N-2}^K} \cdot \frac{N-1}{N}}.$$

Эта вероятность соответствует вероятности выигрыша в случае, если игрок сохраняет первоначальный выбор.

После открытия K дверей остаётся $N - K - 1$ закрытых дверей, отличных от первоначально выбранной. Если игрок решает изменить выбор, он случайным образом выбирает одну из этих дверей. Тогда вероятность выигрыша при изменении выбора равна

$$P_{\text{switch}} = \frac{N - 1}{N} \cdot \frac{1}{N - K - 1}.$$

Пусть p — вероятность того, что игрок сохраняет первоначальный выбор, а $1 - p$ — вероятность того, что он меняет выбор. Тогда итоговая вероятность выигрыша определяется выражением

$$P_{\text{win}} = p \cdot P(A|B) + (1 - p) \cdot (1 - P(A|B)) = p \cdot \frac{1}{C_{N-1}^K} \cdot \frac{1}{N} + (1 - p) \cdot \frac{N - 1}{N} \cdot \frac{1}{C_{N-2}^K}.$$

Полученная формула позволяет исследовать зависимость вероятности выигрыша игрока от параметров N , K и стратегии игрока p .

4 Проверка качества модели

Для проверки модели выполняется вычислительный эксперимент, реализованный при помощи языка программирования Python. Эксперимент повторяется большое количество раз, в нашем случае 10.000 раз, для разных вариантов значений N (от 3 до 10), K (от 1 до $N - 2$) и p (0.0, 0.5 и 1.0), после чего вычисляется доля выигрышей — приближенная вероятность P_{win} , и сравнивается с аналитическим значением, полученным при помощи подстановки значений параметров в исходные формулы. На приведённых ниже итоговых графиках (Рисунки 2, 3 и 4) на осях Y и X указаны значения N и K соответственно, а на их пересечении разница по модулю полученного экспериментально значения с аналитически полученным значением.

При $p_{\text{stay}} = 0$, то есть игрок всегда изменяет свой выбор, наблюдается наиболее высокая точность совпадения теоретических данных с результатами метода Монте-Карло (разница в среднем составляет ≈ 0.002). Это подтверждает классический вывод: чем больше информации (открытых пустых дверей) предоставляет ведущий, тем выгоднее менять первоначальный выбор.

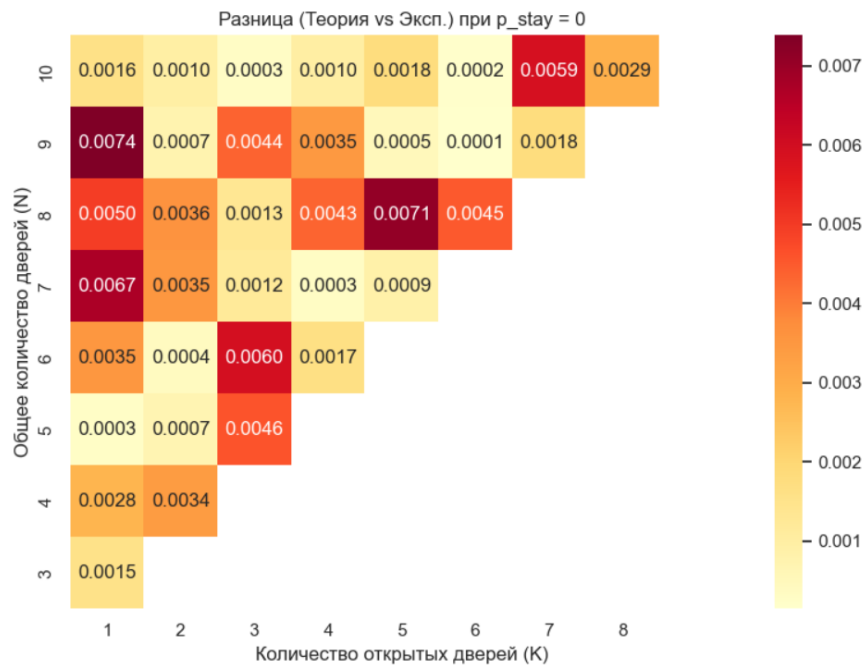


Рис. 2 — Анализ модели при $p = 0$

При $p_{stay} = 0.5$ результаты демонстрируют промежуточные значения. Заметно увеличение статистической погрешности (разница до 0.07) по сравнению со стратегией $p_{stay} = 0$.

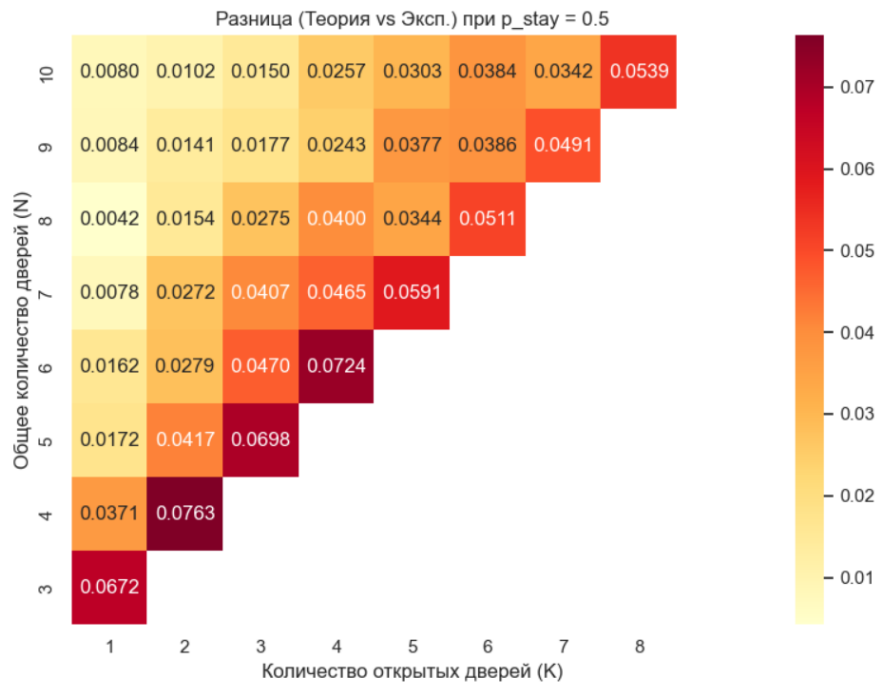


Рис. 3 — Анализ модели при $p = 0.5$

При $p_{stay} = 1$, то есть игрок никогда не изменяет свой выбор, наблюдаются наибольшие отклонения, достигающие 0.1497 при $N = 4$, $K = 2$.

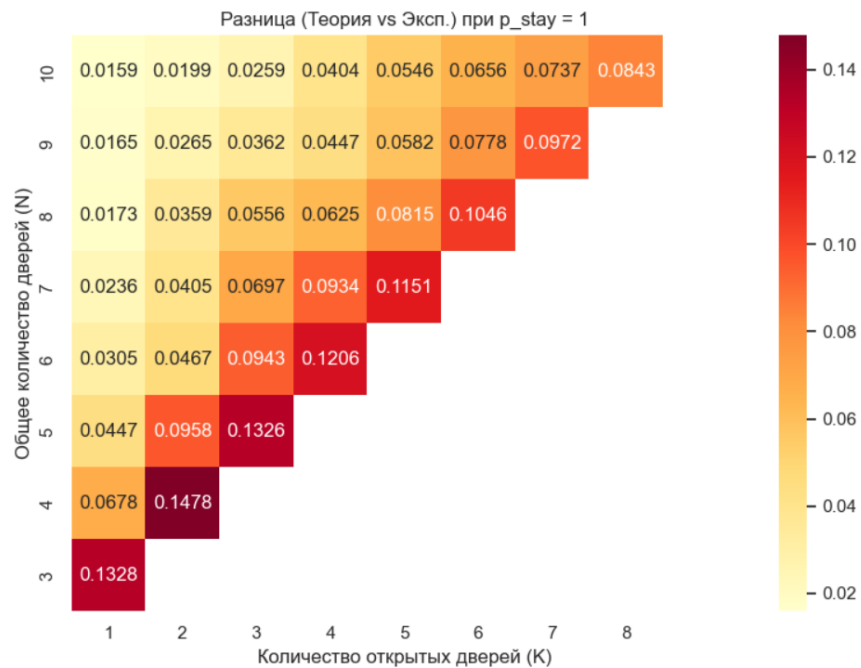


Рис. 4 — Анализ модели при $p = 1$

Анализ построенной вероятностной модели показал высокую адекватность как теоретических расчетов, так и результатов вычислительного эксперимента. Для всех стратегий игрока ($p_{\text{stay}} = 0$, $p_{\text{stay}} = 0.5$ и $p_{\text{stay}} = 1$) наблюдается небольшое отклонение теоретических и экспериментальных вероятностей выигрыша.

Если рассматривать классические значения параметров парадокса Монти Холла, а именно $N = 3$, $K = 1$, $p_{\text{stay}} = 0$, то тут разница с аналитическим решением составляет всего лишь 0.0015, что составляет 15 сотых процента.

Таким образом, проведённый анализ и сравнение результатов вычислительного эксперимента с аналитическим решением демонстрирует, что модель корректно отражает зависимость вероятности выигрыша от параметров N , K и p_{stay} , подтверждая классический парадокс Монти Холла, заключающийся в увеличении вероятности выигрыша при смене выбора игрока после открытия дверей.

5 Вывод

В ходе выполнения домашней работы была построена вероятностная модель обобщённой задачи Монти Холла с N дверями и K пустыми дверями, открываемыми ведущим, а также исследована стратегия игрока, задаваемая параметром p_{stay} — вероятностью сохранения первоначального выбора. Были получены аналитические формулы для вычисления вероятности выигрыша P_{win} игрока с учётом стратегии, количества дверей и числа открытых ведущим пустых дверей, используя формулу Байеса.

Проведён вычислительный эксперимент методом Монте-Карло для проверки адекватности модели. Для всех трёх стратегий ($p_{\text{stay}} = 0, 0.5, 1$) результаты эксперимента хорошо согласуются с теоретическими расчетами. Визуальный анализ графиков показал, что вероятность выигрыша игрока возрастает при увеличении K при смене двери и всегда остаётся относительно низкой при изменении выбора, что подтверждает классический парадокс Монти Холла.

Модель корректно учитывает зависимость вероятности выигрыша от всех параметров (N, K, p_{stay}) и позволяет оценить оптимальную стратегию игрока. Для всех рассматриваемых вариантов стратегия “всегда менять дверь” даёт наибольшую вероятность выигрыша, что согласуется с парадоксом Монти Холла.

6 Практическая реализация

Практическая реализация представлена в листинге 1.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import math
import random

def nCr(n, r):
    return math.comb(n, r)

def theoretical_p_win(N, K, p_stay):
    p_a = 1 / N
    p_not_a = (N - 1) / N
    term_a = (1 / nCr(N - 1, K)) * p_a
    term_not_a = (1 / nCr(N - 2, K)) * p_not_a
    p_a_given_b = term_a / (term_a + term_not_a)
    p_switch = p_not_a * (1 / (N - K - 1))
    return p_stay * p_a_given_b + (1 - p_stay) * p_switch

def monte_carlo_experiment(N, K, p_stay, trials=100000):
    wins = 0
    for _ in range(trials):
        winning_door = random.randint(0, N - 1)
        player_choice = random.randint(0, N - 1)
        possible_to_open = [d for d in range(N) if d != player_choice and d
                             != winning_door]
        opened_doors = random.sample(possible_to_open, K)
        if random.random() < p_stay:
            final_choice = player_choice
        else:
            remaining_doors = [d for d in range(N) if d != player_choice and
                               d not in opened_doors]
            final_choice = random.choice(remaining_doors)
        if final_choice == winning_door:
            wins += 1
    return wins / trials

def plot_heatmaps():
    p_values = [0, 0.5, 1]
    N_range = range(3, 11)

    for p_stay in p_values:
        plt.figure(figsize=(8, 6))
```

```

diff_matrix = np.full((len(N_range), max(N_range) - 1), np.nan)
print(f"\n---                p_stay = {p_stay} ---")
print(f"{'N':<4} | {'K':<4} | {'',                ':<10} | {'',
                ':<12} | {'',                '}')

for i, N in enumerate(N_range):
    for K in range(1, N - 1):
        theory = theoretical_p_win(N, K, p_stay)
        experiment = monte_carlo_experiment(N, K, p_stay)
        diff_matrix[i, K-1] = abs(theory - experiment)
        print(f"{'N':<4} | {'K':<4} | {'theory:.4f'}          | {'experiment:.4f'}
              | {'diff_matrix[i, K-1]:.4f}'")

sns.heatmap(diff_matrix, annot=True, fmt=".4f", cmap="YlOrRd",
            xticklabels=range(1, max(N_range)-1), yticklabels=N_range
            )
plt.gca().grid(False)
plt.gca().invert_yaxis()
plt.title(f'                (                vs                .)                p_stay
          = {p_stay}')
plt.xlabel('                (K)')
plt.ylabel('                (N)')
plt.tight_layout()
plt.show()

if __name__ == "__main__":
    plot_heatmaps()

```

Листинг 1.